

Project Management Software Specification

Project Mentor: *Filip Manev - Git: @Filip.Manev*

Description

This project management software will serve as a simplified task and project management system. The system allows for a hierarchical structure where users are categorized by experience level: **Senior**, **Mid**, and **Junior**, with **Admins** having full system control. A subset of **Senior** users can be designated as **Team Leads**, who are responsible for managing teams and projects.

This project can be divided into 2 sections, the very application visible to every user once they log in, and an Admin version that will be only available to the user designated as an Admin.

Projects represent the largest resource handled by the app. The project is assigned to a Senior Team Lead. The Team Lead has to then assign developers to this project i.e. create a team. Within the project the Team Lead can create tasks which can then be assigned to the team members to complete.

Tasks represent the main working matter in the app. A task (or commonly known as a Ticket) is created to describe the needed action to be taken. These actions can be anything from fixing a small bug to implementing a major feature to the project. The task has multiple statuses that are explained further in this document. When a task is initially created by the team lead it is not assigned to any project member. After member assignment the status of a task can be changed by the members (read carefully the permission hierarchy for task status changes)

Tasks can also receive comments, with full CRUD capabilities of the comment owner.

User Roles and Permissions:

Admin:

Admins have the highest level of control over the system and manage users, projects, tasks, and comments. Admins can:

- Create and manage user accounts.
- Assign user experience levels (Senior, Mid, Junior).
- Designate Seniors as **Team Leads**.
- Create, update, assign, and delete **projects** with information such as:
 - Title
 - Description
 - Requirements
 - Estimated Time of Completion
 - Assignment to a Team Lead
 - Deadline
- View and manage all users, projects, tasks, and comments.
- Full **CRUD** (Create, Read, Update, Delete) access over:
 - User accounts
 - Projects
 - Tasks
 - Comments

**The Admin has full access to every part of the regular application a unique access to the admin panel. Within the regular application the admin has Senior Team Lead clearance to every existing project. Admin users do not need to have a dedicated registration form, you can add them directly in the Database*

All Users:

The following features are required for any user:

- User registration
 - Name/Nickname
 - Email
 - Password (Password in Database has to be hashed)
 - Repeat Password (Password in Database has to be hashed)
 - Level
- User Log in m
 - Name - or - Email
 - Password
- Change password option
 - Enter old password
 - Enter new password

**The Non-Admin users can be created by the Admin through the Admin Panel or can register themselves through a separate registration form. These users will need to be approved by an admin in order for them to be able to log into the app. Users created by the Admin in the Admin Panel are automatically approved and have random password string generated.*

Senior:

Seniors are experienced users and can be split into:

- **Team Lead:** Seniors that are able to lead projects and manage teams.
- **Senior:** Can perform tasks on assigned projects but does not manage teams (does not lead projects).

Team Lead:

- Can be **assigned projects** by Admins.
- Create their own team using users from the **Senior, Mid, and Junior** pools.
- Can create, assign, and manage **tasks** within their assigned project with information:
 - Title
 - Description
 - Time of creation
- Can assign tasks to team members.
- Can change the status of a task to **To Do, In Progress, QA, or Done**
- Can mark the project as **Done** once all tasks are complete.
- Leave comments on any task (including unassigned tasks)

**A Team Lead is defined as a Senior who has been assigned the role of Team Lead by the Admin. An Admin can then assign created projects exclusively to Seniors that have been marked as Team Leads prior. Note that a Team Lead's unique permissions and privileges are scoped (limited) only to the projects that have been assigned to them - meaning - a Team Lead can be a member of a team of a different project as a regular Senior and NOT have the same privileges (adding/removing members to a project) because here he is not the owner i.e. the project was not assigned to him, but has been added by another Team Lead as a regular senior.*

Senior:

- Cannot manage teams (add/remove members to project).
- Can create and manage **tasks** within assigned projects but only once they have been added to a team/project by a **Team Lead**.
- Can change the status of any task.
- Leave comments on any task (including unassigned tasks)

Mid:

Mid-level users have the following permissions:

- Cannot create new tasks, only interact with existing ones.
- Can only assign tasks to themselves or **Junior** users.
- Can change the status of a task **only if it is already assigned to them or any task assigned to a Junior**.
- Can leave comments on tasks they are involved in and tasks that are assigned to Junior.

Junior:

Junior users have the most limited permissions:

- Cannot create new tasks, only interact with tasks already assigned to them by Team Lead/Senior/Mid.
 - Cannot assign tasks to self or to others.
 - Can change the status of a task **only if it is already assigned to them**.
 - Can leave comments only to tasks assigned to themselves.
-

Project and Task Management:

➤ Projects:

- Created by the Admin with details such as title, description, requirements, estimated time of completion, and deadline.
- Projects are assigned to **Team Leads** by Admins.
- Only **Team Leads** can create a team and assign tasks to team members within the project.
- Every Member of the team of a project has access to the project information (defined by the admin during Project creation)

➤ Tasks:

- Tasks are created by **Team Leads or regular Seniors**.
- Tasks have the following statuses:
 - **To Do** (default status when created)
 - **In Progress**
 - **QA**
 - **Done**
- **Team Leads** can assign tasks to any team member (Themselves, Other Seniors, Mid, Junior).
- **Senior** can assign tasks to Themselves, Mid, Junior.

- **Mid** users can assign tasks only to themselves or Junior users.
- **Junior** users can only update task statuses assigned to them.

➤ **Comments:**

- Created tasks have a comment section
- Comments must have a timestamp of the time of creation
- Seniors (Team Leads and Non-Team Leads) can leave comments on any task, regardless if the task has/has not received an assignee.
- Mid can leave comments on tasks already assigned to them and to tasks assigned to Juniors
- Junior can leave comments only on tasks assigned to them
- Regardless of the User level, every comment owner can Edit and Delete their own comments
 - Edited comments should be marked as edited
 - No need to keep timestamps of edits, only on first comment creation
- Within the admin panel, an Admin has the clearance to full CRUD operations to any comment of any user
- **Automatic Comments** on task status change, for example:
 - **[USERNAME]** changed the status from **To Do** to **In Progress** (timestamp is handled same as a regular comment insertion)

➤ **Teams:**

- Teams represent the group of users that are assigned to a Project.

**The handling of what a Team is in your project is more liberal, meaning you can choose what a Team represents within your project. A Team can be the group of Users that have been added to a project by a Team Lead - or - you can treat Teams as a separate entity/resource within your App and Database. If you go on this route then the Senior Team Lead should have a separate tool to create Teams and assign (connect) them to a Project that has been assigned to him.*

Bonus:

UI Feedback Implementation

Implement intuitive UI feedback throughout the application to enhance the user experience. This includes:

- Displaying success and error messages for CRUD operations and form submissions.
 - Showing loading indicators during actions like task status updates or project assignments.
 - Providing real-time feedback for user interactions to ensure seamless and responsive navigation.
 - Handling edge cases, such as invalid input or server errors, with clear and informative messages.
-

Technology Stack:

- **HTML/CSS** (Bootstrap/Tailwind can be used if preferred).
 - **PHP (OOP preferred)** for server-side logic, handling CRUD operations for users, projects, tasks, and comments.
 - **JavaScript (jQuery)** for interactive UI elements.
 - **SQL** for storing user data, projects, tasks, comments, and statuses.
-

This project management software provides a clear hierarchy of permissions, with responsibilities distributed among **Admins**, **Senior Team Leads**, **Seniors**, **Mid-level**, and **Junior** users. By building this system, students will gain hands-on experience with role-based access control, CRUD operations, and database interactions, while also reinforcing core web development skills.

System Permission Hierarchy

 = Yes  = No	Assigns Tasks To Members	Creates Tasks (Tickets)	Creates Teams/Adds Users to Project	Creates Projects	Access To Admin Panel
Admin					
Senior (Team Lead)					
Senior (Non-Team Lead)					
Mid					
Junior					

Task Delegation Hierarchy

 = Yes  = No	To Self	To Junior	To Mid	To Senior
Senior (Team Lead)				
Senior (Non-Team Lead)				
Mid				
Junior				

Project Goals & Objectives:

- The student will verify his/her understanding and knowledge regarding PHP, SQL and JavaScript
- The Student will gain firm grasp on role/permission based systems

Project Step-by-Step Procedural Information:

Step 1: Attend the project brief

Step 2: Thorough review of the project specification

Step 3: Plan an approach: what should be done first, construct an ER Diagram.

Step 4: Setup a repository and create a development branch

Step 5: Start work on whatever fits your vision best and build from there

Step 6: Be sure to work on major segments on a separate branch, merge this branch to dev or main when you are done with the implementation

Step 7: Be sure to test your implementation from multiple angles

Step 8: Attend the Project mentorship session for progres checkup and consultation

Step 9: Make proper documentation and include it on the project repository

Step 10: Add your project mentor & reviewer as a maintainer to the project repository before 23:59 of the deadline day

Project Deliverables

A short textual description is included for how to run the project locally (maybe in the *README.md* file of the project). The project itself, with all the files needed for running it locally, is available on GitLab. Additionally, you should add the GitLab repository link in the Brainster portal and also, enable access to the person who will be reviewing your project.

Evaluation system

Criteria - 20pts	Excellent 20p	Proficient 15p	Good 10p	Fair 5p
Functionality	<i>Fully implemented CRUD operations on users, projects, tasks, and comments. AJAX used for all operations. Intuitive interface with smooth workflows.</i>	<i>CRUD operations mostly functional, some AJAX applied. Good user experience with minor improvements needed.</i>	<i>Basic CRUD operations implemented, limited AJAX. Interface is functional but lacks refinement.</i>	<i>Partial CRUD functionality, minimal or no AJAX usage. Admin panel is usable but needs significant improvements.</i>
Task Creation & Assignment	<i>Team Leads and Seniors can create and assign tasks properly. Task statuses are fully operational, with correct assignment permissions across all user roles.</i>	<i>Task creation and assignment are mostly correct, but occasional issues with role-based assignments or status changes.</i>	<i>Tasks can be created and assigned but lack full functionality (e.g., limited status updates or permission control).</i>	<i>Task creation and assignment are minimally functional, with major issues in assigning tasks based on user roles.</i>
Comment System	<i>Fully functional comment system with CRUD capabilities for all roles. Comments are timestamped, and edited comments are clearly marked. On changing of task status an auto comment is inserted.</i>	<i>Comment system is mostly functional with some issues (e.g., missing timestamps or minor bugs in CRUD operations).</i>	<i>Comment system implemented with basic functionality but lacks features like editing indicators or robust CRUD handling.</i>	<i>Basic comment functionality exists, but CRUD operations are limited, and timestamps may be missing.</i>

Criteria - 10pts	Excellent 10p	Proficient 7.5p	Good 5p	Fair 2.5p
User Role & Permissions Handling	<i>All roles (Admin, Senior, Mid, Junior) have proper permissions, with Team Leads receiving unique privileges within their projects. Role-based access control is well-implemented and secure.</i>	<i>Most roles have correct permissions, but minor issues with role access or handling of Team Leads.</i>	<i>Roles implemented with some permission issues (e.g., Senior, Mid, or Junior not fully differentiated). Team Lead role partially functional.</i>	<i>Roles and permissions are loosely handled, with significant security or functionality concerns.</i>

Project Management	Admins can create, update, and assign projects to Team Leads. Projects are fully linked with teams and tasks, with proper deadlines and details visible to all users.	<i>Project management mostly functional, but some minor issues in team creation or task linking.</i>	<i>Projects can be created and assigned, but details or team linking is incomplete or buggy.</i>	<i>Basic project creation exists but lacks full functionality for assignments and team integration.</i>
User Authentication	<i>Secure and fully functional user authentication, including registration, login, and logout. Passwords are hashed, and security best practices (SQL Prepared Statements)</i>	<i>User authentication works well, but lacks certain security features (e.g., weaker hashing, basic session handling). Minor issues with authentication flow.</i>	<i>Basic authentication is implemented but may have security flaws (e.g., plain-text passwords) or incomplete features like logout.</i>	<i>User authentication is poorly implemented or incomplete, with security vulnerabilities or broken functionality.</i>
Form Validation	<i>Comprehensive client-side and server-side form validation implemented. All user inputs are properly validated, including edge cases (e.g., invalid email, special characters). Error messages are user-friendly and validation is seamless.</i>	<i>Form validation works on both client-side and server-side, but with minor inconsistencies or missed edge cases. Error messages could be clearer.</i>	<i>Basic form validation implemented, but lacks coverage for some inputs or edge cases. Some validation happens only on server-side or validation messages are not user-friendly.</i>	<i>Minimal or incomplete form validation, allowing erroneous data or bad user experience.</i>
Code Structure and Organization	<i>Clean, modular code following best practices (OOP, DRY, MVC) with clear documentation.</i>	<i>Code is well-structured, with minor redundancy or lack of documentation.</i>	<i>Code works but is not modular or lacks clear organization.</i>	<i>Code is disorganized, with poor readability or structure.</i>

Bonus Criteria - 10pts	Excellent 10p	Proficient 7.5p	Good 5p	Fair 2.5p
Bonus: UI Feedback Implementation	<i>Fully implement intuitive UI feedback, success/error messages, loading spinners, ensuring responsive interactions..</i>	<i>Good UI feedback for most actions, with minor inconsistencies or limited edge case handling.</i>	<i>Basic UI feedback implemented for major actions but lacks polish or edge case handling.</i>	<i>Minimal or incomplete feedback, leading to user confusion in some actions.</i>

Deadline

5 weeks after its presentation, at 23:59 (end of the day).

Assessment Rules

- ❖ Fair Assessment: ethical considerations
 - Assessors should ensure that the assessments are conducted in a fair and ethical manner, respecting the principles of academic integrity and honesty.
- ❖ Reliability and Validity: enabling consistency
 - Assessments should be consistent and reliable, meaning that they yield consistent results when applied repeatedly to the same task or performance.
 - Assessments must accurately gauge the knowledge, skills, or abilities they are intended to evaluate, ensuring their validity as indicators.
- ❖ Feedback: as a method for continuous improvement
 - Assessors should offer constructive feedback that identifies strengths and areas for improvement. The Feedback should be specific and actionable, it should include thought provoking guides and should challenge the student to become better at a specific task.
- ❖ Late Submission Policy: assessing assignments after their deadline
 - Students should be allowed a grace period of 3 days (72 hours) to make a late submission on any assignment, with the notice that they will be deducted 20% from the total possible points.
- ❖ Plagiarism Policy: assessing assignments with matching solutions
 - In the event of suspected plagiarism, the assessor is required to promptly collaborate with the Student Experience Coordinator/Team as the initial step. Together, they will draft a notice to remind students of the strict prohibition against plagiarism, with potential repercussions for recurrent violations. The following actions will be considered in cases of repeated plagiarism:
 - If a submitted solution exhibits substantial similarity, exceeding 60%, with another student's work (individually or within a group), the respective challenge/project will incur a 50% reduction from the maximum points attainable.
 - In cases where a solution is identified as more than 90% identical to another student's work (individually or within a group), the project/challenge in question will receive a score of 0 points.
 - The use of generative AI is encouraged as a learning tool in our educational programs; however, students must engage with the material and contribute with original thought. Reliance on AI for complete content generation is strictly prohibited and will result in point deductions, official warning, or other academic penalties.
 - Upon completion of the assessment process for each challenge/project, the assessor is tasked with selecting the most complete, optimal, and creative solution and to showcase it by publishing it on the platform together with the assessment results.
- ❖ Timeliness: timeframe for delivering results and feedback to students
 - Feedback on projects should be provided within 14 days after the deadline has passed (depending on the project's complexity).